

Docker Compose 개념 학습

⚙ 상태	진행 중
☰ 다중 선택	

핵심

- **services:** 컨테이너 정의 단위 (zookeeper, kafka, kafka-ui 각각 하나씩)
- **depends_on:** 컨테이너 실행 순서 제어 (kafka는 zookeeper가 먼저 떠야 함)
- **environment:** 컨테이너 내부 환경변수로 설정 주입
- **ports:** 호스트-컨테이너 포트 매핑
- **networks:** 컨테이너 간 통신용 (같은 네트워크에 있어야 서로 hostname으로 접근 가능)

참고

- [\[Docker\] Docker 이해하기 -4 : Docker Compose 이해하고 구성하기](#)
- [Docker Compose + Apache Kafka, Kafka-UI 환경 구성 방법](#)

1. Docker 핵심 개념

1.1 Image와 Container

- Image : 코드 실행에 필요한 환경을 통째로 패키징한 것
- Container : Image의 실행 환경, Image의 실행된 인스턴스
- Image를 실행(run)시키면
 - 이미지 → 벽돌(틀)
 - 컨테이너 → 실제 집(운영중)

1.2 나의 Image를 온라인 저장소에 업로드할 때

1. **Dockerfile(레시피)** : 도커는 Dockerfile를 읽어서 하나의 완성된 Image(환경 패키지)를 만든다
2. **docker build** : Dockerfile 읽어서 → 나의 로컬에서 Image 생성
 - ex) 로컬에 `myimage:latest` 이미지가 생김
3. **docker push** : Image를 Registry(온라인 저장소)에 업로드
 - 내 컴퓨터에서 만든 `myimage:latest` 이미지를 → 온라인 저장소(Docker Hub 등)에 올린다.

```
docker push myrepo/myimage:latest
```

- 이제 누구나 `docker pull` 로 이 image 환경 패키지를 다운로드 받을 수 있다.
- Registry(온라인 저장소) 예시
 - Docker Hub
 - AWS ECR
 - GitHub Container Registry
 - GCP Artifact Registry

1.3 다른 사람의 Image 다운로드해서 실행할 때

1. `docker pull` : Registry에서의 Image를 내 컴퓨터로 다운로드

- 다른 사람의 Image

```
docker pull myrepo/myimage:latest # Image 다운로드
```

2. `docker run` : Image 실행(run) → Container(실행 환경) 생성

- 생성된 Container에서 Image 실행
- 즉, 컨테이너는 Image의 **실행된 인스턴스(instance)**

```
docker run myrepo/myimage:latest # "컨테이너 환경" 생성되고 Image 실행
```

이 순간:

- 이미지 → 벽돌(틀)
- 컨테이너 → 실제 집(운영중)

2. Docker Compose

- Docker Compose를 사용하는 이유

기존의 Dockerfile을 통해 '단일 컨테이너'를 생성-관리-배포하는 과정은 비교적 간단했지만, '여러 컨테이너'를 일괄적으로 띄우려면 시간 소요가 많기 때문에 `docker compose` 를 사용

- Docker Compose 의 정의

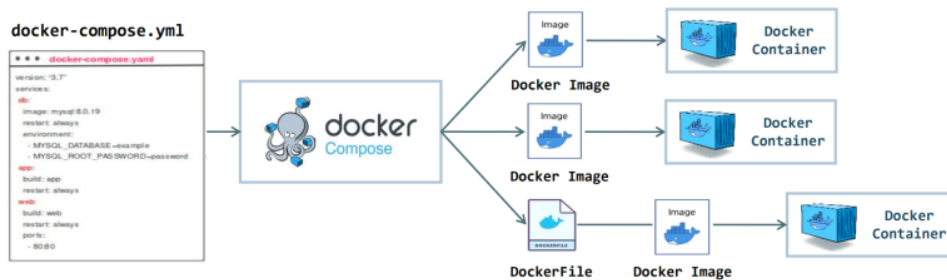
하나의 Compose 파일(YAML/YML)로, 여러 컨테이너를 실행/관리한다

- 장점

서비스, 네트워크 및 볼륨 같은 애플리케이션 서비스를 구성할 수 있습니다.

또한 개발, 테스트, 스테이징, 운영 및 CI 워크플로우 환경에서 애플리케이션을 이동하여 배포가 가능하기에 관리가 매우 편리합니다.

2.1 Dockerfile과 docker-compose.yml파일



```

# docker-compose.yml

services:
  db:
    image: mysql:8.0.19      # ① 이미 완성된 이미지 사용 → Dockerfile 불필요
    restart: always
    environment:
      - MYSQL_DATABASE=example
      - MYSQL_ROOT_PASSWORD=password

  app:
    build: app              # ② ./app 폴더 안의 Dockerfile을 찾아서 "build(로컬에서 이미지 생성)"
    restart: always

  web:
    build: web              # ③ ./web 폴더 안의 Dockerfile을 찾아서 "build"
    restart: always
    ports:
      - "80:80"
  
```

- 컨테이너가 만들어지는 경로
 - db 서비스: `image: mysql:8.0.19` → 이미 만들어진 Docker Image를 바로 가져와서 → Docker Container 실행
 - app 서비스: `build: app` → **DockerFile** → Docker Image → Docker Container
 - web 서비스: `build: web` → **DockerFile** → Docker Image → Docker Container
 핵심은 db는 Dockerfile이 필요 없고, app/web은 Dockerfile을 거친다는 것
- Dockerfile과 docker-compose.yml의 역할 차이

	Dockerfile	docker-compose.yml
역할	이미지를 어떻게 만들지 정의	컨테이너들을 어떻게 실행하고 연결할지 정의
범위	컨테이너 1개(=이미지 1개)의 내부	여러 컨테이너 전체의 오케스트레이션
예시 내용	"이 OS 위에 이 언어 런타임 깔고, 이 코드 복사하고, 이 명령어로 실행해라"	"이 이미지들을 이 포트로, 이 순서로, 이 환경변수로 띄워라"

- `image: mysql:8.0.19` 처럼 이미 Docker Hub에 있는 완성 이미지를 쓸 땐
→ Dockerfile이 아예 필요 없고, 만들어진 이미지를 실행해서 컨테이너를 띄운다
- `build: app` 이라고 쓰면

→ compose가 자동으로 `./app/ Dockerfile` 을 찾아서 그 안의 명령어대로 이미지를 빌드하고, 빌드된 이미지를 실행해서 컨테이너를 띄운다

즉, Dockerfile은 이미지 1개를 어떻게 만들지 (레시피),
docker-compose.yml은 여러 서비스(컨테이너)를 어떻게 조합/실행할지 (설계도)

2.2 Docker Compose 처리 과정 요약

1. Dockerfile로 이미지를 만드는 방법을 정의한다

(Define how to build the image in a Dockerfile)

- Dockerfile 안에 "어떤 베이스 이미지를 쓸지, 어떤 파일을 복사할지, 어떤 명령어로 실행할지"를 정의
- `compose` 파일에서 `build:` 를 쓰는 서비스에 한해 필요한 단계
- `image:` 로 완성된 이미지를 그대로 쓰는 서비스는 이 단계 자체가 필요 없음

2. Compose 파일(YAML/YML) 안에서 여러 서비스(컨테이너)들을 정의하고, 그 관계를 설정한다

(Define services and their relationships in a Compose file)

- 여기서 처음으로 "서비스"라는 단위가 등장
 - db, app, web처럼 각 컨테이너를 하나의 서비스로 선언한다
- 이 파일에서 정의하는 것
 - 각 서비스가 `image` 를 쓸지 `build` 를 쓸지,
 - 포트 매핑, 볼륨 마운트, 네트워크, 환경 변수는 어떻게 할지

3. `docker-compose up` 명령어를 사용하여 애플리케이션(일괄 서비스)을 시작한다

(Use Docker Compose to start the application)

- Compose 파일에 정의된 모든 서비스를 시작
- 필요한 Docker 이미지가 로컬에 없으면 자동으로 다운로드하고, 컨테이너를 띄운다

+ 서비스 상태 확인

- `docker-compose ps` 명령어로 현재 실행 중인 서비스 상태를 확인할 수 있다

+ 서비스 중지

- `docker-compose stop` : 컨테이너를 멈추기만 함 (삭제 X)
- `docker-compose down` : 모든 서비스를 중지하고 컨테이너, 네트워크, 볼륨까지 삭제

2.3 Docker Compose 명령어

Docker Compose 명령어	설명
<code>docker-compose up</code>	Docker Compose 파일에 정의된 모든 서비스를 시작합니다. 필요한 이미지를 자동으로 다운로드하고 컨테이너를 생성, 시작합니다.
<code>docker-compose down</code>	실행 중인 모든 서비스를 중지하고 컨테이너, 네트워크, 볼륨을 제거합니다.
<code>docker-compose ps</code>	Docker Compose에 의해 관리되는 현재 실행 중인 서비스의 상태를 보여줍니다.
<code>docker-compose start</code>	이전에 생성된 서비스를 시작합니다.

Docker Compose 명령어	설명
<code>docker-compose stop</code>	실행 중인 서비스를 중지합니다.
<code>docker-compose build</code>	서비스를 빌드합니다.
<code>docker-compose logs</code>	서비스의 로그를 출력합니다.
<code>docker-compose pull</code>	서비스에 필요한 모든 이미지를 다운로드합니다.
<code>docker-compose restart</code>	서비스를 재시작합니다.

2.4 주요 속성

- 최상위 속성
 - `version` : docker compose 파일의 버전을 지정
 - 버전은 파일의 구조를 결정
 - `services` : docker 컨테이너의 목록을 정의
 - 각 서비스는 각 컨테이너의 구성을 설명
 - `networks` : 컨테이너가 사용할 네트워크 지정
 - `volumes` : 호스트와 컨테이너 간의 볼륨 매핑을 지정
 - `configs` : Swarm mode 에서 사용되는 설정을 지정

2.4.1 services 속성

- 하나의 `docker-compose.yml` 안에 여러 컨테이너를 정의하는 구조 이해
- 각 서비스가 독립된 컨테이너로 뜬다는 것, `image` vs `build` (Dockerfile로 직접 빌드)의 차이
- 실습: zookeeper, kafka, kafka-ui 서비스 3개가 어떻게 분리되어 정의되는지 yml 뜯어보기
- services 속성의 하위 속성
 - `image: mysql:8.0.19`
이미 Docker Hub에 있는 완성 이미지를 쓸 땐 → Dockerfile이 아예 필요 없고, 만들어진 이미지를 실행해서 컨테이너를 띄운다
 - `build: app`
compose가 자동으로 `./app/ Dockerfile` 을 찾아서 그 안의 명령어대로 이미지를 빌드하고, 빌드된 이미지를 실행해서 컨테이너를 띄운다
 - `depends_on` : 해당 서비스가 시작하기 전에 시작되어야 하는 다른 서비스를 지정
 - `depends_on`에 지정한 서비스는 해당 서비스보다 먼저 실행된다
 - 컨테이너 **시작 순서**만 제어하고, 내부 서비스가 완전히 "준비됨"을 보장하진 않는다는 점이 중요 포인트
 - 예: zookeeper 컨테이너는 띄웠지만 아직 2181 포트 리스닝 전인데 kafka가 붙으려다 실패하는 경우 있음
 - 심화 개념: `healthcheck` + `condition: service_healthy` 옵션 (실제 운영에서는 이걸로 순서 보장)
 - `environment` : Docker 컨테이너에 전달될 환경 변수를 지정
 - `environment`로 지정한 환경 변수는 `env_file`로 지정한 파일에 있는 변수를 덮어쓴다
 - 컨테이너 안 프로세스에 환경변수를 주입하는 방식

- Kafka 이미지의 경우 `KAFKA_` 접두사 붙은 환경변수들이 실제 `server.properties` 설정값으로 변환된다는 원리 (confluentinc 이미지 특징)
- `KAFKA_ADVERTISED_LISTENERS` 가 뭔지 꼭 이해하고 넘어가기 — 컨테이너 외부(호스트)에서 접속할 때 브로커가 자기 자신을 어떤 주소로 광고하는지에 대한 설정이라, 나중에 EC2 배포할 때 이 값 때문에 연결 안 되는 이슈 자주 남
 - `env_file` : docker 컨테이너에서 사용할 환경 변수를 포함하는 파일의 경로를 지정
 - `ports`
 - "호스트포트:컨테이너포트" 매핑 문법
 - 컨테이너 내부 네트워크와 호스트 네트워크가 분리되어 있다는 개념부터 이해해야 이 매핑이 왜 필요한지 감이 올
 - `networks` : 이 서비스가 연결할 네트워크를 지정
 - `links` : 이 서비스와 연결할 다른 서비스를 지정
 - links에 지정한 서비스는 해당 서비스와 같은 네트워크에 연결된다

2.4.2 networks 속성

```
# docker-compose.yml

# 최상위 레벨 (networks)
networks:
  my-network:          # ← 여기서 네트워크를 "정의/생성"
    driver: bridge

# 최상위 레벨 (services)
services:
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    networks:          # 서비스 레벨 ← 여기서 그 네트워크를 "사용/연결"
      - my-network

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    networks:          # 서비스 레벨 ← 같은 네트워크에 연결해야 서로 hostname으로 통신
      - my-network
```

위치	역할
최상위 <code>networks:</code>	"이런 네트워크가 존재한다"를 선언/생성하는 곳 (네트워크 이름, driver 종류 등 설정)
<code>services.<서비스명></code> <code>></code> <code>.networks:</code>	그 서비스를 어떤 네트워크에 연결할지 지정하는 곳

- 최상위 `networks:` = "이런 이름의 도로를 새로 만든다"
- `services` 안의 `networks:` = "이 컨테이너를 그 도로에 연결한다"

특징

- 각 서비스에서 명시적으로 `networks:` 안 써도 `docker-compose`가 기본 네트워크를 자동 생성해서 같은 파일 안의 서비스들을 묶어준다
 - 안 써도 되는 경우
 - `docker-compose up` 하면 `compose`가 "`프로젝트폴더명_default`"라는 이름의 네트워크를 알아서 만들고, 같은 `yml` 파일 안의 모든 서비스를 자동으로 그 네트워크에 연결시켜준다
 - 컨테이너 간에는 서비스 이름(`kafka` , `zookeeper`)이 곧 `hostname`으로 동작한다는 게 핵심 — `KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181` 에서 `zookeeper` 가 바로 이 원리
 - 그래서 최상위/서비스 레벨 둘 다 안 써도 `zookeeper:2181` 처럼 서비스 이름으로 접속이 되는 거예요.
 - 명시적으로 쓰는 경우
 - 여러 개의 네트워크로 서비스를 분리하고 싶을 때 쓴다

```
# docker-compose.yml

networks:
  backend:      # DB, Kafka 등 내부 통신용
  frontend:     # 웹서버 등 외부 노출용

services:
  db:
    networks: [backend]          # backend만 접근 가능
  kafka:
    networks: [backend]
  web:
    networks: [backend, frontend] # 둘 다 접근 가능 (중간 다리 역할)
```

- 이렇게 하면 `'web'`은 `'db'`, `'kafka'`에 접근 가능하지만,
- `'frontend'`에만 있는 다른 서비스는 `'db'`에 직접 접근을 못 한다. 즉, 격리 가능

2.4.3 volumes 속성

서비스(컨테이너) 간에 데이터를 공유하고 서비스를 다시 시작할 때 데이터를 유지하기 위해 사용

- 도커 컨테이너에 영구적인 데이터를 저장하거나, 컨테이너 간에 데이터를 공유하는 데 필요한 디스크 공간을 할당
- 컨테이너의 파일 시스템에 직접 접근하여 데이터를 수정하거나 컨테이너 외부에서 데이터를 사용할 수 있습니다.
- 서비스에서 각각 필요한 볼륨을 생성하고 관리할 수 있습니다.

- **volumes**
 - **'db-data'**라는 도커 볼륨을 생성

```
# 최상위 레벨 (volumes)

volumes:
  db-data:
```

- **services**

- 'db-data'라는 도커 볼륨은 'backend'와 'backup' 서비스에서 공유하여 사용

```
# 최상위 레벨 (services)

services:
  backend: # backend 서비스
    image: example/database
    volumes:
      - db-data:/etc/data

  backup: # backup 서비스
    image: backup-service
    volumes:
      - db-data:/var/lib/backup/data
```

- **backend 서비스**

- 'example/database'라는 이미지를 기반으로 컨테이너를 생성
- 'volumes' 설정을 통해 'db-data'라는 볼륨을 컨테이너의 '/etc/data' 디렉터리에 마운트

- **backup 서비스**

- 'backup-service'라는 이미지를 기반으로 컨테이너를 생성
- 'db-data' 볼륨을 컨테이너의 '/var/lib/backup/data' 디렉터리에 마운트

▼ 심화 공부 : depends_on의 한계와 ADVERTISED_LISTENERS

두 개 다 실행하다가 제일 많이 막히는 지점

1. depends_on의 한계

될 보장하고, 될 안 보장하는가

```
kafka:
  depends_on:
    - zookeeper
```

이건 딱 하나만 보장해요: "**zookeeper 컨테이너를 먼저 start 시켜라**". 그게 끝이에요.

보장 안 하는 것: zookeeper 컨테이너 안에서 실제로 Zookeeper 프로세스가 완전히 뜨고, 2181 포트가 리스닝 상태가 됐는지는 전혀 확인 안 해요.

실제로 벌어지는 일

```
0초: docker-compose up 실행
0초: zookeeper 컨테이너 "시작됨" (컨테이너 생성 완료, but 내부 프로세스는 이제 막 부팅 시작)
0초: kafka 컨테이너도 거의 동시에 "시작됨" (depends_on 조건 충족했다고 판단)
1~3초: zookeeper 내부에서 JVM 뜨고, 설정 읽고, 2181 포트 리스닝 시작 (아직 진행 중)
0.5초: kafka가 벌써 zookeeper:2181로 연결 시도 → Connection Refused
```


컨테이너가 "떴다"는 것과 그 안의 애플리케이션이 "준비됐다"는 건 완전히 다른 타이밍이에요. 특히 Zookeeper나 Kafka처럼 JVM 기반 프로그램은 부팅에 몇 초씩 걸리는데, `depends_on` 은 그 몇 초를 기다려주지 않아요.

왜 그런데도 실습에서는 대체로 잘 됐나

Kafka 클라이언트 라이브러리 대부분이 **연결 실패 시 자동 재시도** 로직을 갖고 있어서, 몇 초 안에 재시도하다 보면 결국 불긴 해요. 그래서 로컬 실습에서는 운 좋게 넘어가는 경우가 많은데, **컨테이너 리소스가 부족하거나(느린 PC), CI/CD 파이프라인처럼 딱딱한 환경**에서는 이 타이밍 문제가 실제 장애로 튀어나와요.

제대로 하려면: healthcheck + condition

```
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  healthcheck:
    test: ["CMD", "nc", "-z", "localhost", "2181"]
    interval: 5s
    timeout: 3s
    retries: 5

kafka:
  depends_on:
    zookeeper:
      condition: service_healthy # ← "떴다"가 아니라 "healthcheck 통과했다"를 기다림
```

`healthcheck` 가 "진짜로 2181 포트가 응답하는지"를 주기적으로 검사하고, `condition: service_healthy` 는 그 검사를 통과할 때까지 kafka 시작을 미뤄줘요. 이게 진짜 순서 보장이예요.

2. KAFKA_ADVERTISED_LISTENERS

이건 "네트워크가 컨테이너/호스트로 분리돼 있다"는 개념과 바로 연결되는 부분이에요.

문제 상황을 먼저 이해하기

Kafka는 좀 특이한 프로토콜이에요. 클라이언트가 브로커에 접속하면, 브로커가 "나(브로커)는 사실 이 주소로도 접근 가능해"라고 **자기 자신의 주소를 클라이언트에게 알려줘요**. 클라이언트는 그 알려준 주소로 실제 데이터를 주고받아요.

```
클라이언트 → 브로커: "연결 좀 할게"
브로커 → 클라이언트: "OK, 근데 나는 사실 kafka:9092 주소야. 앞으로 여기로 얘기해"
클라이언트: kafka:9092로 재접속 시도
```

문제가 터지는 지점

브로커가 "나는 kafka:9092야"라고 광고하는데, 이게 컨테이너 내부에서만 통하는 주소예요. **로컬 PC에서 실행하는 카프카 클라이언트(예: 콘솔 프로듀서, kafka-ui 말고 별도 로컬 앱) 입장에선 `kafka` 라는 hostname을 몰라요** (호스트 PC는 Docker 내부 DNS를 모르니까요).

```
[호스트 PC의 내 앱] → "kafka:9092? 그런 주소 몰라" → 연결 실패
```

그래서 이렇게 설정하는 거예요

```
environment:
```

`KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092`

이건 "브로커야, 클라이언트한테 네 주소 알려줄 때 `localhost:9092` 라고 말해"라는 뜻이에요. 그러면 호스트 PC에서 실행하는 앱은 `localhost:9092` 로 접속 가능하니 문제 없이 붙어요.

컨테이너끼리 통신할 땐 어떻게 되나

여기서 재밌는 지점인데, `kafka-ui` 처럼 같은 Docker 네트워크 안에 있는 컨테이너가 kafka에 붙을 때는

`KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092` 처럼 **서비스 이름(`kafka`)**을 씁니다. `localhost` 를 쓰면 오히려 안 돼요 — kafka-ui 컨테이너 입장에서 `localhost` 는 "kafka-ui 자기 자신"을 가리키는 거지 kafka 컨테이너를 가리키지 않으니까요.

즉 접속하는 주체가 호스트나 컨테이너냐에 따라 필요한 주소가 다른데, `ADVERTISED_LISTENERS`는 하나만 광고하다 보니 상황에 따라 꼬이는 거예요.

리스너를 여러 개 쓰는 실전 설정 (나중에 EC2 배포할 때 필요해짐)

```
kafka:
  environment:
    KAFKA_LISTENERS: INTERNAL://0.0.0.0:9092,EXTERNAL://0.0.0.0:9093
    KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:9092,EXTERNAL://localhost:9093
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
  ports:
    - "9093:9093"
```

- **INTERNAL (9092):** 같은 Docker 네트워크 안 컨테이너끼리 쓰는 통로 → `kafka:9092` 로 광고
- **EXTERNAL (9093):** 호스트나 외부에서 접속하는 통로 → `localhost:9093` 으로 광고 (실제 EC2에서는 여기서 `<EC2 공인 IP>:9093` 으로 바꿉니다)

정리

상황	ADVERTISED_LISTENERS에 뭘 써야 하나	실패하면 나는 증상
로컬 PC 앱 → 컨테이너 kafka	<code>localhost:9092</code>	<code>Connection to node -1 could not be established</code>
다른 컨테이너(kafka-ui) → kafka	서비스 이름 <code>kafka:9092</code>	kafka-ui에서 브로커 목록이 비거나 타임아웃
나중에 EC2 배포 시 외부 → kafka	EC2 공인 IP/도메인	로컬에선 잘 됐는데 배포하면 갑자기 연결 안 되는 전형적 이슈

지금 로컬 실습 단계에서는 `localhost:9092` 하나로 충분한데, 다음 주 AWS 배포 넘어갈 때 이 개념이 그대로 이슈로 터질 거예요. 그때 다시 짚어드릴게요.

▼ 심화 공부 : ports 와 hostname

1. 왜 포트 매핑이 필요한가 — 네트워크가 분리되어 있다는 것부터

Docker는 컨테이너 하나마다 독립된 가상 네트워크 공간을 만들어요. 이게 핵심이에요.

```
[호스트 PC (내 맥북/윈도우)] ← 이것도 하나의 네트워크
|
| (분리되어 있음, 서로 안 보임)
```

```
[Docker 내부 네트워크] ← 컨테이너들끼리만 있는 별도 세계
├─ zookeeper 컨테이너 (2181 포트로 리스닝 중)
├─ kafka 컨테이너 (9092 포트로 리스닝 중)
└─ kafka-ui 컨테이너 (8080 포트로 리스닝 중)
```

kafka 컨테이너 안에서 "나는 9092 포트 열어놨어!"라고 해도, 그건 **Docker 내부 네트워크 안에서만 열려있는 것**이지, 내 컴퓨터(호스트)에서 브라우저로 localhost:9092 쳐봤자 원래는 안 열려요. 호스트와 컨테이너는 서로 다른 네트워크 세계에 있으니까요.

2. 그래서 ports 매핑이 하는 일

```
ports:
  - "8080:8080"
```

이건 "**호스트포트:컨테이너포트**" 문법인데, 풀어 쓰면:

"내 컴퓨터(호스트)의 8080번 포트로 누가 접속하면, 그걸 컨테이너 내부의 8080번 포트로 그대로 넘겨줘라" (일종의 포트 포워딩)

그래서 이렇게도 매핑할 수 있어요 (숫자가 달라도 됨):

```
ports:
  - "3000:8080" # 호스트에서는 3000번으로 접속해야, 컨테이너 내부 8080으로 연결됨
```

이러면 브라우저에서 localhost:3000 쳐야 kafka-ui가 열려요. localhost:8080 으로 쳐도 안 열립니다 — 호스트 쪽 포트가 3000으로 지정됐으니까요.

즉, ports: 는 "**호스트 → 컨테이너**"로 가는 문을 하나 뚫어주는 것이라고 이해하시면 돼요. 이게 없으면 컨테이너는 자기들끼리만 통신하고, 내 컴퓨터(호스트)에서는 아예 접근할 방법이 없어요.

3. hostname 개념 — 컨테이너끼리는 얘기가 다름

방금 건 "**호스트 ↔ 컨테이너**" 얘기였고, 이번엔 "**컨테이너 ↔ 컨테이너**" 얘기예요. 완전히 다른 상황이라 나눠서 봐야 해요.

```
[Docker 내부 네트워크] ← 컨테이너들은 이 안에서 이미 같은 동네에 삽니다
├─ zookeeper (hostname: "zookeeper")
└─ kafka (hostname: "kafka")
```

같은 compose 파일 안의 서비스들은 이미 같은 **Docker 네트워크 안에 있어서**, 서로 접근할 때는 포트 매핑이 필요 없어요. 대신 서비스 이름 자체가 그 컨테이너의 **hostname(별명 같은 것)**으로 자동 등록돼요.

그래서:

```
kafka:
  environment:
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
```

이 줄은 "zookeeper라는 이름의 컨테이너, 그 안의 2181번 포트로 접속해라"라는 뜻이에요. zookeeper 가 IP 주소도 아니고 localhost 도 아닌데 접속이 되는 이유가, Docker가 내부적으로 "**zookeeper**"라는 이름 = 그 컨테이너의 실제 IP로 자동 변환(DNS 역할)해주기 때문이에요.

정리 — 언제 뭘 쓰는지

상황	뭐가 필요한가	예시
호스트(내 PC) → 컨테이너	ports: 매핑 필수	브라우저에서 kafka-ui 켜보기: localhost:8080
컨테이너 → 다른 컨테이너	ports: 필요 없음, 서비스 이름 = hostname	kafka가 zookeeper 찾아갈 때: zookeeper:2181

즉 kafka-ui의 ports: "8080:8080" 은 내가 브라우저로 들여다보기 위한 문이고, kafka가 zookeeper에 붙는 zookeeper:2181 은 컨테이너끼리 이미 같은 동네에 살아서 이름만으로 부르는 것이예요. 이 둘을 헷갈리지 않는 게 포인트예요.